

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	1
ВВЕДЕНИЕ	2
1. ОБЗОР ТЕКУЩЕГО СОСТОЯНИЯ ПРЕДМЕТА ИССЛЕДОВАНИЯ.....	4
1.1 Обзор зарубежных исследований.....	4
1.2 Обзор отечественных исследований.....	5
1.3 Общая характеристика проблемы исследования	7
2. ОБЗОР ТЕХНОЛОГИЙ И МЕТОДОВ РЕШЕНИЯ ПРОБЛЕМЫ.....	9
2.1 Основные методы решения проблемы и их характеристики	9
2.2 Основные технологии решения проблемы и их характеристики	10
2.3 Алгоритм выбора методов и технологий решения проблемы.....	12
3 ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К РЕШЕНИЮ ПРОБЛЕМЫ	17
3.1 Общая характеристика предлагаемого решения	17
3.2 Функциональные требования к предлагаемому решению	20
3.3 Нефункциональные требования к предлагаемому решению	23
ЗАКЛЮЧЕНИЕ.....	28
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	31

ВВЕДЕНИЕ

Актуальность настоящего исследования обусловлена широким распространением сложных человеко-машинных интерфейсов, требующих оперативного анализа визуальной информации и принятия решений в условиях дефицита времени и высокой когнитивной нагрузки. Примером такой среды является киберспорт, в частности, многопользовательская онлайн-игра Dota 2. Однако практическая значимость работы выходит далеко за рамки игровой индустрии. Основная идея исследования заключается в разработке и апробации универсального подхода к созданию систем ситуационного анализа, основанных на технологии захвата и обработки видеопотока с экрана в реальном времени с помощью предварительно обученной модели искусственного интеллекта.

Традиционные системы поддержки принятия решений часто полагаются на структурированные данные из API или специализированных датчиков, что ограничивает область их применения. В отличие от них, предлагаемый подход использует в качестве входных данных визуальный интерфейс (скриншот), что позволяет работать с любым приложением, не требующим глубокой интеграции на уровне кода или протоколов. Ключевой вызов при этом заключается в обеспечении высокой скорости и точности анализа сложной, динамически меняющейся визуальной сцены.

Целью данной работы является исследование и формализация требований к программному решению, способному в реальном времени захватывать изображение с экрана, интерпретировать его с помощью нейросетевой модели, извлекать семантически значимые сущности и контекст, а затем на основе этой информации и предварительно заложенных знаний выдавать пользователю рекомендации. Dota 2 выбрана в качестве удобной и репрезентативной тестовой среды для валидации подхода благодаря своей сложности, чётким правилам и доступности данных для обучения модели.

Для достижения поставленной цели требуется провести анализ существующих методов и технологий захвата экрана, компьютерного зрения и машинного обучения для задач интерпретации динамических сцен. Необходимо исследовать подходы к созданию и использованию базы знаний для генерации контекстно-зависимых рекомендаций. Также важно сформулировать архитектурные, функциональные и нефункциональные требования к системе, обеспечивающие её работоспособность с минимальной задержкой и высокой надёжностью.

Практическая значимость работы заключается в создании методологического и технического базиса для разработки ассистирующих систем нового класса. Подобные системы потенциально могут быть применены в областях, где требуется анализ действий пользователя в сложных интерфейсах в реальном времени: в тренажёрах и симуляторах (авиационных, медицинских), в системах мониторинга диспетчерских пунктов, в качестве интеллектуальных помощников для работы со специализированным программным обеспечением (например, биржевыми терминалами, CAD-системами), а также для создания адаптивных обучающих сред. Таким образом, данное исследование направлено на решение актуальной научно-технической задачи создания гибких, неинвазивных систем анализа на основе компьютерного зрения и ИИ.

1. ОБЗОР ТЕКУЩЕГО СОСТОЯНИЯ ПРЕДМЕТА ИССЛЕДОВАНИЯ

1.1 Обзор зарубежных исследований

Анализ зарубежных научных публикаций и технической документации свидетельствует о значительном внимании научного и инженерного сообщества к проблематике обработки визуальной информации в реальном времени. В работах, представленных на ведущих конференциях, таких как CVPR, ICCV и NeurIPS, основное внимание уделяется совершенствованию архитектур нейронных сетей для задач детекции, классификации и семантической сегментации изображений с акцентом на повышение скорости вывода без существенной потери точности. Например, в исследованиях группы FAIR (Facebook AI Research) представлены архитектуры детекции объектов Detectron2, оптимизированные для производительности. Работы в области моделей YOLO (You Only Look Once) от Джозефа Редмона и его коллег посвящены созданию однопроходных детекторов, достигающих высоких скоростей, что критически важно для систем реального времени. В области эффективного захвата экрана и создания оверлеев значительный вклад вносят практические разработки сообщества вокруг таких технологий, как OBS Studio, где глубоко проработаны методы низкоуровневого захвата через DirectX и Vulkan. Исследования NVIDIA в рамках технологии NVFBC и NVIFR направлены на аппаратно-ускоренный захват видеопотока с минимальным влиянием на производительность основного приложения. Ключевым вызовом, обсуждаемым в современных зарубежных исследованиях, остается проблема создания высокопроизводительных, низколатентных и надежных систем, интегрирующих захват экрана, анализ на основе ИИ и отрисовку поверх изображения, особенно в контексте ресурсоемких приложений и игр.

1.2 Обзор отечественных исследований

В рамках отечественной научной школы также ведется активная и плодотворная работа по широкому направлению компьютерного зрения и смежных технологий для систем реального времени. Российские исследователи и инженеры, представляющие как ведущие академические институты, так и передовые технологические компании, вносят весомый и признанный вклад как в фундаментальные теоретические аспекты этих дисциплин, так и в их практическое, прикладное воплощение. В частности, научные коллективы и лаборатории, базирующиеся в таких уважаемых учебных и исследовательских центрах, как Московский государственный университет имени М.В. Ломоносова (МГУ), Университет ИТМО, Московский физико-технический институт (МФТИ) и Сколковский институт науки и технологий (Сколтех), регулярно публикуют результаты изысканий, формирующие актуальный ландшафт отечественной науки в данной области. Например, работы крупной лаборатории компьютерной графики и мультимедиа, функционирующей на базе МГУ под руководством авторитетного профессора А.С. Кремлева, системно затрагивают актуальные вопросы, связанные с эффективной потоковой обработкой, сжатием и последующим семантическим анализом непрерывных видеопотоков в условиях ограниченных вычислительных ресурсов. Параллельно, интенсивные исследования, проводимые в ИТМО в сфере передовых методов машинного обучения для задач обработки изображений, целенаправленно фокусируются на инновационной разработке новых, более эффективных архитектур глубоких нейронных сетей, оптимизации алгоритмов их обучения и адаптации для работы на специфических типах данных.

Кроме академических институций, значительный вклад в развитие отрасли вносят крупные российские технологические компании, которые не только применяют существующие наработки, но и активно инвестируют в собственные научные исследования. Корпорации уровня «Яндекс» и

«ABBYY» регулярно публикуют в открытых и профессиональных источниках результаты своих изысканий, посвященных тонкой оптимизации алгоритмов компьютерного зрения для решения конкретных, востребованных бизнес-задач. В фокусе их внимания находятся такие проблемы, как высокоточное распознавание текста (OCR) в неструктурированных документах и сложных естественных сценах, надежная детекция и классификация разнородных объектов на изображениях, а также сегментация сцен. Также безусловно стоит отметить наличие в России конкурентоспособных, а в некоторых случаях и передовых, разработок в узкоспециализированной области алгоритмов технического (промышленного) зрения, где требования к детерминированному реальному времени, робастности и точности являются абсолютно ключевыми. Эти решения находят применение в системах автоматизированного контроля качества на производственных линиях, в роботизированных комплексах и других задачах промышленной автоматизации.

Однако при всем многообразии проводимых исследований, в значительно более узкой и специфической технологической нише, ориентированной именно на создание программных решений для интерактивного, низколатентного захвата и последующего интеллектуального анализа содержимого экрана персонального пользовательского компьютера с немедленным выводом контекстно-зависимых интеллектуальных подсказок посредством графического наложения (оверлея), ситуация выглядит иначе. Комплексных, фундаментальных научных исследований, посвященных именно архитектурным и системным аспектам построения таких комплексных конвейеров «захват-анализ-визуализация» в отечественной открытой научной литературе представлено сравнительно меньше. Большинство практически применимых отечественных разработок в этой области носят закрытый, коммерческий или внутрикорпоративный характер, их архитектура и ключевые решения не публикуются и не становятся предметом широкого научного обсуждения. Именно этот факт формирует актуальное и перспективное поле для проведения собственного, независимого

исследования, целью которого является не повторение закрытых паттернов, а построение открытой, хорошо документированной и модульной эталонной архитектуры. Такая архитектура могла бы служить прозрачным образцом для научного сообщества, основой для обучения и платформой для будущих открытых инноваций в этом перспективном сегменте разработки программного обеспечения.

1.3 Общая характеристика проблемы исследования

Исходя из проведенного анализа различных источников и данных, проблема состоит в отсутствии целостного, универсального и технически изящного подхода к разработке программных систем, способных функционировать в режиме реального времени. Идеальный комплекс такого рода должен одновременно и непрерывно выполнять три взаимосвязанные задачи: бесшовно захватывать видеопоток с экрана, производить его семантический анализ с использованием предобученных моделей искусственного интеллекта для извлечения структурированной информации и, наконец, накладывать результаты этого анализа на исходное изображение в виде интерактивной графической подложки - оверлея. При этом критически важным условием является минимальная задержка на всех этапах обработки, обеспечивающая эффект «живого» и немедленного отклика системы.

Актуальность проблемы заключается в том, что сейчас для этого нет готового инструмента. Нужно вручную соединять три разные технологии, которые изначально друг с другом не работают. Это сложно, долго и результат часто получается медленным и нестабильным. Современные технологические компоненты, как правило, охватывают лишь отдельные аспекты этой сложной цепочки. Специализированные библиотеки для захвата экранного контента не обладают встроенными механизмами для интеллектуального анализа. Мощные фреймворки для машинного обучения, в свою очередь, не адаптированы к специфичным требованиям конвейеров реального времени,

работающих с потоковым видеовводом с экрана. Одновременно с этим, системы для отрисовки графических оверлеев могут создавать конфликты с другим программным обеспечением, особенно с полноэкранными приложениями и играми, либо неоптимально использовать ресурсы центрального и графического процессоров, приводя к потере производительности.

Следовательно, возникает объективная и насущная потребность не в простом объединении разрозненных инструментов, а в глубоком системном подходе. Такой подход должен охватывать все стадии жизненного цикла программного комплекса: от первоначального проектирования совместимой и эффективной архитектуры, через этап корректной интеграции разнородных технологических компонентов, до их тонкой взаимной оптимизации. Реализация подобного подхода позволит создавать принципиально новые классы программного обеспечения — высокоэффективные интерактивные помощники и аналитические инструменты. Эти системы смогут находить применение в самых различных областях: от автоматизированного тестирования пользовательских интерфейсов и создания иммерсивных обучающих систем до профессиональной аналитики игрового процесса и разработки вспомогательных технологий для людей с ограниченными возможностями.

2. ОБЗОР ТЕХНОЛОГИЙ И МЕТОДОВ РЕШЕНИЯ ПРОБЛЕМЫ

2.1 Основные методы решения проблемы и их характеристики

Для решения обозначенной проблемы на уровне алгоритмов и подходов применяется спектр методов:

1. Методы захвата экрана. К ним относятся:

- Захват на уровне растрового образа всего экрана или окна через Windows GDI. Характеризуется простотой реализации, но низкой производительностью и высокой загрузкой CPU.
- Захват через DirectX/Graphics Capture API (Windows). Позволяет с минимальными затратами получать кадры из буфера видеокарты, поддерживает захват поверхностей Direct3D. Характеризуется высокой скоростью и низкой нагрузкой на систему.
- Захват с использованием аппаратного ускорения (например, NVIDIA NVFBC). Обеспечивает максимальную производительность с минимальным влиянием на рендеринг основного приложения, но требует специализированного оборудования и драйверов.

2. Методы анализа изображений с помощью ИИ. Основными являются:

- Свёрточные нейронные сети для классификации изображений. Позволяют определить общий контекст или состояние системы, отображаемой на экране. Отличаются сравнительно высокой скоростью работы, но предоставляют обобщенную информацию.

- Нейронные сети для детекции объектов (R-CNN, YOLO, SSD). Позволяют локализовать и классифицировать множество объектов на одном кадре. Обеспечивают детальную информацию, но более требовательны к вычислительным ресурсам, что может влиять на частоту кадров в конвейере реального времени.
- Методы семантической сегментации. Присваивают каждому пикселю изображения класс, что полезно для выделения сложных областей. Наиболее ресурсоемки и редко применяются в системах с жесткими требованиями к задержке.

3. Методы вывода графического оверлея. Включают:

- Создание поверх всех окон прозрачного окна с отключенным захватом мыши. Простой метод, но может конфликтовать с полноэкранными приложениями и античит-системами.
- Внедрение в конвейер рендеринга целевого приложения через перехват вызовов графических API (DirectX/OpenGL хуки). Обеспечивает надежный вывод поверх любого контента, включая полноэкранные 3D-приложения, но является сложным в реализации и потенциально нестабильным.
- Использование аппаратно-ускоренного наложения через функции драйверов видеокарт (например, NVIDIA Game Filter). Высокая производительность и стабильность, но требует поддержки со стороны драйвера и ограничена в возможностях кастомизации.

2.2 Основные технологии решения проблемы и их характеристики

На технологическом уровне реализация указанных методов опирается на различные программные стеки и инструменты:

1. Технологии для модуля захвата экрана:

- Windows API (GDI, DXGI): Набор системных интерфейсов для работы с графикой. DXGI предоставляет методы для эффективного захвата буфера обмена и поверхностей Direct3D. Характеризуется прямой интеграцией с ОС, хорошей производительностью, но привязанностью к платформе Windows.
- OBS Studio (libobs): Открытая кроссплатформенная библиотека для захвата и стриминга. Включает высокооптимизированные плагины для захвата через DirectX, OpenGL, Vulkan. Характеризуется высокой надежностью и производительностью, может быть использована как внешняя библиотека.
- FFmpeg: Мультимедийный фреймворк, способный захватывать экран через различные backends (gdigrab, dshow, dxgigrab). Характеризуется кроссплатформенностью, но может уступать в производительности и простоте интеграции в приложения реального времени специализированным решениям.

2. Технологии для модуля анализа ИИ:

- Фреймворки глубокого обучения (TensorFlow, PyTorch): предоставляют полный цикл инструментов для обучения и развертывания моделей. PyTorch часто предпочитают для исследований, TensorFlow — для продакшена. Для интеграции в C++ приложения используют TensorFlow C++ API или PyTorch LibTorch. Характеризуются богатой экосистемой и сообществом.
- ONNX Runtime: Кроссплатформенный движок для выполнения моделей в формате ONNX. Позволяет обучать модель в одном фреймворке (PyTorch), конвертировать в ONNX и выполнять с высокой производительностью на разных аппаратных

платформах (CPU, GPU) через единый API. Характеризуется высокой оптимизацией и простотой интеграции.

- OpenCV DNN модуль: Библиотека компьютерного зрения, включающая модуль для загрузки и выполнения предобученных моделей из различных фреймворков. Удобна для быстрого прототипирования и использования в связке с другими функциями OpenCV для предобработки изображений.

3. Технологии для модуля вывода оверлея:

- DirectX/OpenGL: Низкоуровневые графические API, позволяющие отрисовывать геометрию и текст поверх существующей сцены. Требуют глубоких знаний в компьютерной графике. Для DirectX удобно использовать фреймворки типа Direct2D для отрисовки 2D+ элементов.
- Фреймворки для создания графических интерфейсов (Qt, WinForms, WPF): позволяют создавать прозрачные окна. Qt, в частности, предоставляет возможность создания полупрозрачных и нерегулярных окон. Однако такие окна могут не отображаться поверх полноэкранных приложений в режиме исключительного владения (exclusive fullscreen).
- Библиотеки для внедрения оверлеев (Overlay Toolkit, собственные реализации через хуки): Специализированные решения, часто используемые в игровых софтверных оверлеях (например, Steam Overlay, Discord Overlay). Их реализации обычно закрыты, но принципы работы документированы в сообществе разработчиков.

2.3 Алгоритм выбора методов и технологий решения проблемы

На основании детального и всестороннего анализа существующих подходов, методологий и технологических стеков был разработан и структурирован алгоритм выбора оптимальных методов и технологий, представляющий собой четкую, логически обоснованную и воспроизводимую последовательность действий. Данный алгоритм призван служить практическим руководством для архитекторов и разработчиков, позволяя на системной основе принимать взвешенные технические решения при проектировании подобных комплексов, минимизируя риски и обеспечивая достижение требуемых эксплуатационных характеристик.

Первоначальным и ключевым шагом, закладывающим фундамент для всех последующих выборов, является детальное и формализованное определение целевых характеристик и ограничений будущей системы. Необходимо точно установить, на какой целевой аппаратно-программной платформе (например, современные версии Windows 10 или 11 с 64-битной архитектурой) будет работать решение, а также четко и измеримо сформулировать комплекс требований к производительности. К этим требованиям относятся критически важная целевая частота захвата и последующего анализа кадров, измеряемая в кадрах в секунду (FPS), максимально допустимая пользователем или предметной областью задержка полного цикла обработки «захват-анализ-вывод», которую необходимо измерить от момента фиксации кадра на экране до момента отрисовки на нем итоговой подсказки, а также глубокая квалификация характера анализируемого визуального контента — является ли он преимущественно статичным, как текстовые интерфейсы или дашборды, или же представляет собой высокодинамичные 3D-сцены с быстрым изменением. Дополнительно на этом этапе задаются и документируются целевые качественные показатели точности и необходимой детализации семантического анализа: достаточно ли

для решения задачи простой классификации всего изображения по категориям, требуется ли точная детекция с локализацией множества объектов или же необходима пиксельная сегментация для выделения сложных областей. Параллельно оценивается и устанавливается допустимый уровень нагрузки на ключевые вычислительные ресурсы — центральный (CPU) и графический (GPU) процессоры, что напрямую влияет на выбор ресурсоемких технологий.

Исходя из этих сформулированных критериев и ограничений, осуществляется обоснованный выбор конкретного метода и технологии для модуля захвата экрана. При работе с целевыми приложениями, которые являются полноэкранными играми или профессиональным софтом, активно использующим графические API DirectX или OpenGL, технологический приоритет должен быть отдан высокопроизводительным низкоуровневым технологиям, таким как DXGI (DirectX Graphics Infrastructure) или специализированному аппаратно-ускоренному захвату через механизмы вроде NVIDIA ShadowPlay. Если же основным требованием является кроссплатформенность (работа под Linux, macOS) или система нацелена на захват обычных окон рабочего стола без сложной графики, то в качестве альтернатив рассматриваются более универсальные, но часто менее производительные методы, вроде классического GDI в Windows или соответствующих кроссплатформенных библиотек, таких как средства захвата в составе фреймворка Qt или библиотеки X11 на Linux. Тем не менее, для достижения максимальной производительности, минимальной задержки и высокой стабильности в рамках экосистемы Microsoft Windows оптимальным и рекомендуемым решением является именно технология DXGI Desktop Duplication API.

Следующим важным этапом алгоритма является системный подбор методов и технологий для модуля анализа изображений с помощью искусственного интеллекта. На основе ранее определенных и зафиксированных требований к детализации и точности выбирается базовый

тип решаемой задачи ИИ: это может быть классификация всего кадра, детекция объектов с выделением bounding boxes или семантическая сегментация. После определения типа задачи осуществляется подбор конкретной, оптимальной для данных условий архитектуры сверточной нейронной сети, например, семейства YOLO для задач детекции в реальном времени, которое обеспечивает наилучший известный баланс между скоростью работы инференса и точностью распознавания. Для эффективной и производительной интеграции выбранной модели в высокопроизводительное нативное приложение, написанное, к примеру, на C++, рекомендуется использование определенного, проверенного стека технологий: первоначальное обучение или тонкая настройка модели осуществляется в удобной для исследований среде PyTorch, ее последующий экспорт в универсальный, оптимизированный формат обмена ONNX (Open Neural Network Exchange), и, наконец, выполнение вывода с помощью высокооптимизированного движка ONNX Runtime с задействованием возможностей GPU для максимального ускорения вычислений.

Далее, на основе понимания условий эксплуатации, определяется подходящий метод вывода графического оверлея с результатами анализа. Если целевое анализируемое приложение работает в стандартном оконном или псевдополноэкранном режиме, часто бывает достаточно и технически проще создать прозрачное окно с высоким приоритетом отображения, располагающееся поверх других окон средствами оконного менеджера ОС. Однако для гарантированного и бесперебойного отображения информационных элементов поверх современных полноэкранных 3D-приложений и игр, которые часто используют монопольный режим работы с графическим адаптером, требуется применение более сложных и низкоуровневых подходов. К ним относится, например, внедрение в графический конвейер рендеринга самого приложения через перехват системных вызовов графических API DirectX или OpenGL (так называемые

хуки) или использование специализированных закрытых библиотек, предоставляемых некоторыми производителями. На начальных этапах прототипирования и для упрощения первичной реализации можно рассмотреть вариант вывода данных в отдельном, всегда поверх остальных, окне, которое вручную позиционируется пользователем поверх основного окна целевого приложения.

Заключительная, интегрирующая фаза алгоритма полностью посвящена вопросам системной интеграции и глубокой оптимизации выбранных компонентов в единый рабочий конвейер. Здесь происходит детальное проектирование общей архитектуры данных, форматов их представления и потоков выполнения, направленное на минимизацию задержек и предотвращение блокировок, например, за счет организации асинхронной очереди кадров и параллельного выполнения задач захвата, анализа ИИ и отрисовки оверлея в отдельных, синхронизированных между собой потоках. Затем проводится тщательное, поэтапное профилирование каждого звена созданного конвейера с использованием специализированных инструментов (профайлеров CPU/GPU) для выявления, локализации и последующего устранения узких мест. Процесс оптимизации носит итеративный, циклический характер и может включать широкий спектр мер, таких как постепенное снижение разрешения захватываемого изображения для уменьшения объема данных, применение методов квантования весов модели ИИ для значительного ускорения ее работы при несущественной потере точности, а также низкоуровневую оптимизацию алгоритмов рендеринга самого графического оверлея для снижения нагрузки на GPU.

3 ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К РЕШЕНИЮ ПРОБЛЕМЫ

3.1 Общая характеристика предлагаемого решения

В качестве предлагаемого решения рассматривается программный комплекс, предназначенный для демонстрации принципов работы системы анализа экранного контента в реальном времени. Данный комплекс является воплощением рассмотренных ранее теоретических подходов и призван решить ключевую проблему создания высокопроизводительного конвейера, способного обеспечивать минимальную задержку между захватом визуальной информации, её интеллектуальным анализом и предоставлением пользователю контекстно-зависимых данных через графический интерфейс. Разрабатываемое решение не ставит своей целью узкую специализацию на одной конкретной задаче, а фокусируется на построении универсальной, расширяемой архитектуры, которая может служить платформой для последующей разработки широкого спектра прикладных инструментов, начиная от интерактивных обучающих систем и заканчивая профессиональными средствами мониторинга и аналитики программных интерфейсов.

Архитектурно решение будет построено по строгому модульному принципу с четким разделением ответственности на три основных, логически независимых компонента: модуль захвата экрана, модуль анализа изображений с помощью модели искусственного интеллекта и модуль визуализации результатов. Такой подход позволяет не только упростить процесс разработки и отладки каждого компонента по отдельности, но и обеспечивает высокую степень гибкости и сопровождаемости системы в целом. В дальнейшем это открывает возможности для легкой замены одной из технологических реализаций (например, смены метода захвата или движка для выполнения модели ИИ) без необходимости глобальной переработки всего кодового

основания, а также для параллельного развития отдельных модулей разными специалистами.

Модуль захвата экрана будет выступать в роли источника данных для всей системы. Его основная и критически важная задача — периодическое получение растровых изображений (скриншотов) с заданной пользователем частотой. Источником для захвата может служить как весь экран целиком, что актуально для системного мониторинга или анализа взаимодействия между несколькими приложениями, так и строго указанная пользователем прямоугольная область, что позволяет сфокусироваться на конкретном элементе интерфейса и значительно снизить объем обрабатываемых данных. Этот модуль должен быть реализован с использованием наиболее высокопроизводительных API, предоставляемых операционной системой, что позволит достичь минимальной нагрузки на вычислительные ресурсы и исключить появление артефактов или пропусков кадров, особенно при работе с ресурсоемкими приложениями, использующими аппаратное ускорение. Для обеспечения плавности работы всего конвейера и предотвращения блокировок, полученные изображения будут немедленно помещаться в специально организованный буфер обмена между потоками выполнения. Этот буфер, организованный по принципу очереди, служит для асинхронной передачи кадров от модуля захвата к модулю анализа, что позволяет первому продолжать работу по сбору данных, не дожидаясь, пока второй завершит обработку предыдущего кадра.

Модуль анализа изображений является интеллектуальным ядром всего комплекса. Его функционал строится вокруг предобученной модели искусственного интеллекта, способной распознавать заранее определенные паттерны, объекты, текстовые элементы или специфические состояния интерфейса на предоставленных скриншотах. На этапе инициализации данный модуль загружает модель из внешнего файла, что обеспечивает возможность ее обновления без пересборки приложения. Получив новый кадр из межмодульного буфера, модуль анализа выполняет над ним обязательный

цикл предобработки. Этот цикл включает в себя изменение размера изображения до формата, ожидаемого входным слоем нейронной сети, конвертацию цветового пространства (например, из BGR в RGB) и нормализацию числовых значений пикселей, что является стандартной практикой для подготовки данных к подаче в модель глубокого обучения. После подготовки изображение передается на вход модели ИИ для выполнения инференса — процесса вычислительного вывода. Полученные от модели «сырые» данные, которые могут представлять собой координаты bounding box'ов, метки классов, коэффициенты уверенности или маски сегментации, проходят этап интерпретации. На этом этапе они преобразуются в структурированный, понятный для последующих модулей формат, например, в набор объектов с четко определенными свойствами (тип, координаты, уверенность), который уже готов к визуализации.

Модуль визуализации выступает в роли финального звена в цепочке обработки, ответственного за непосредственное взаимодействие с конечным пользователем. Его задача — получать структурированные результаты от модуля анализа и отрисовывать их на экране поверх всего остального графического контента в виде интуитивно понятного графического наложения (оверлея). Для этого модуль должен поддерживать отрисовку разнообразных графических элементов: текстовых подсказок и аннотаций, стрелок, указывающих направление внимания или последовательность действий, выделяющих прямоугольников, обводящих области интереса, иконок, обозначающих статусы или категории, а также других индикаторов. Данный модуль должен обеспечивать высокую производительность отрисовки, чтобы не вносить дополнительную задержку в конвейер, и обладать устойчивостью, гарантируя, что оверлей будет корректно отображаться поверх полноэкранных приложений, включая современные 3D-игры.

Ожидается, что данное решение, интегрирующее в себе все три описанных компонента, позволит в полной мере продемонстрировать жизнеспособность и эффективность полного цикла работы системы реального

времени — от сбора визуальных данных до предоставления интеллектуальной обратной связи. Практическая ценность разрабатываемого комплекса заключается не только в его непосредственном функционале, но и в том, что он послужит надежной и хорошо документированной основой, эталонной архитектурой для разработки более специализированных прикладных решений. На его базе могут быть созданы интерактивные помощники для освоения сложного программного обеспечения, системы контроля и аудита действий пользователя в корпоративных средах, обучающие тренажеры с адаптивными подсказками или инструменты автоматизированного тестирования графических интерфейсов. Таким образом, работа над данным проектом закладывает фундамент для целого семейства полезных программных продуктов.

3.2 Функциональные требования к предлагаемому решению

Функциональные требования к системе четко описывают, какие конкретные действия она должна выполнять. Они охватывают четыре ключевых аспекта: захват экранного контента, его обработку, визуализацию результатов и общее управление. Детальное определение данных требований является фундаментальным этапом проектирования, поскольку именно на их основе будет оцениваться корректность работы программного комплекса и его соответствие поставленным целям. Каждое требование должно быть сформулировано таким образом, чтобы его выполнение можно было объективно проверить в процессе тестирования готового продукта, что исключает неоднозначности и субъективные трактовки при приемке работы.

В части захвата экранного контента система обязана предоставить пользователю гибкий и интуитивно понятный выбор источника. Это может быть захват всего экрана, что актуально для анализа общих системных процессов или работы с приложениями, занимающими несколько мониторов.

Также должна быть реализована возможность захвата отдельного окна любого приложения, работающего в операционной системе, что позволяет сфокусировать анализ на конкретной программе и минимизировать объем обрабатываемых данных. Кроме того, необходима функция захвата произвольной прямоугольной области, задаваемой пользователем вручную, что обеспечивает максимальную гибкость для узкоспециализированных задач, где важна только определенная часть интерфейса. Независимо от выбора, система должна стабильно работать, в том числе с приложениями, запущенными в полноэкранном режиме, что является критически важным условием для анализа современных игр или профессионального софта, использующего весь экран. Кроме того, необходима возможность тонкой настройки частоты захвата, например, в диапазоне от 1 до 30 кадров в секунду, для баланса между производительностью и детализацией анализа. Низкая частота снижает нагрузку на систему и подходит для наблюдения за медленно меняющимися интерфейсами, в то время как высокая частота необходима для отслеживания быстрой динамики в реальном времени. Реализация механизма захвата должна учитывать особенности работы графического драйвера и операционной системы для минимизации артефактов и потери кадров.

Далее, система должна уметь обрабатывать и анализировать полученные изображения, что составляет ядро ее интеллектуальной функциональности. Для этого ей требуется загружать предобученную модель искусственного интеллекта из внешнего файла стандартного формата, что обеспечивает модульность и возможность обновления модели без перекомпиляции всего приложения. Перед анализом каждый захваченный кадр автоматически проходит обязательный этап предобработки, который включает изменение размера изображения до разрешения, ожидаемого моделью, что является необходимым условием для корректной работы сверточных нейронных сетей. Также выполняется конвертация цветового пространства, например, из BGR в RGB или в оттенки серого, в зависимости от требований модели, и нормализация данных, при которой значения пикселей приводятся к

определенному диапазону для повышения стабильности и скорости обучения. Затем система выполняет инференс модели — процесс вывода, при котором на основе входного изображения вычисляется результат. Этот процесс может запускаться либо на каждом захваченном кадре для максимальной оперативности, либо с заданным пользователем интервалом для снижения вычислительной нагрузки. Полученные «сырые» результаты, которые могут представлять собой многомерные тензоры с координатами ограничивающих рамок объектов, метками классов, вероятностями принадлежности или масками сегментации, система должна интерпретировать в четкий, структурированный и машиночитаемый формат, такой как JSON или внутренние структуры данных, готовый для последующего отображения и использования другими модулями.

Третий блок требований касается визуализации, которая является пользовательским интерфейсом системы и напрямую влияет на удобство восприятия информации. Результаты анализа должны отображаться поверх всех остальных окон в виде графического наложения, что обеспечивает их постоянную видимость и не требует от пользователя переключения между окнами. Системе необходимо поддерживать отрисовку стандартных графических элементов, включая текст для вывода текстовых подсказок и меток, прямоугольники для выделения областей интереса, стрелки для указания направления или последовательности действий, линии для соединения элементов или построения графиков, и иконки для визуального обозначения статусов или категорий объектов. Пользователь должен иметь возможность гибко настраивать их внешний вид, изменяя цвет, что важно для цветовой дифференциации разных типов информации, прозрачность для ненавязчивого отображения поверх важного контента, размер для адаптации к различным разрешениям экрана и шрифт для улучшения читаемости текстовых элементов. Критически важно, чтобы все эти элементы корректно позиционировались на экране, будучи привязанными к тем областям, которые были проанализированы, что требует точного сопоставления координат на

исходном скриншоте с координатами на экране пользователя с учетом возможных изменений масштаба и разрешения.

Наконец, система должна быть управляемой и настраиваемой, что предполагает предоставление пользователю полного контроля над ее работой. Для этого требуется предоставить отдельный пользовательский интерфейс, например, окно настроек, которое может быть вызвано по горячей клавише или из системного трее. Через этот интерфейс можно сконфигурировать все параметры захвата, включая выбор источника, частоту кадров и область интереса, выбрать и загрузить модель ИИ из файловой системы, а также задать внешний вид оверлея, сохранив профили настроек для быстрого переключения между сценариями использования. У пользователя должна быть возможность запускать и останавливать процесс анализа в реальном времени одной командой, например, с помощью кнопки в интерфейсе или назначенной комбинации клавиш, что позволяет оперативно контролировать работу системы. Также необходима функция включения и выключения отображения самого оверлея без необходимости остановки фоновых процессов захвата и анализа, что позволяет оперативно скрывать или показывать информацию в зависимости от текущих задач пользователя, не прерывая сбор данных, что может быть полезно для временного освобождения экрана или проведения сравнительных тестов.

3.3 Нефункциональные требования к предлагаемому решению

Нефункциональные требования описывают свойства системы, определяющие не ее конкретные функции, а качественные характеристики, такие как производительность, надежность, удобство и эффективность. Эти требования не менее важны, чем функциональные, поскольку непосредственно влияют на практическую пригодность системы, удовлетворенность пользователя и возможность ее интеграции в реальные

рабочие процессы. Они задают количественные и качественные рамки, в которых должно функционировать решение, и являются ключевыми критериями при оценке его успешности на этапе тестирования и внедрения.

1. Производительность:

- полная задержка конвейера, представляющая собой суммарное время от момента отображения кадра на экране пользователя до момента отрисовки оверлея на основе анализа именно этого кадра, не должна превышать 500 миллисекунд при заданной частоте захвата 5 кадров в секунду. Данное требование обусловлено необходимостью обеспечения интерактивности обратной связи, чтобы подсказки оставались актуальными по отношению к быстро меняющейся ситуации на экране. Превышение этого порога задержки может привести к тому, что выводимая информация будет устаревшей и вводящей в заблуждение. Для достижения этого показателя необходима глубокая оптимизация каждого этапа конвейера: захвата, передачи данных между модулями, выполнения модели ИИ и отрисовки графики.
- система должна поддерживать стабильную частоту захвата и анализа, заданную в конфигурации пользователем, без проседаний более чем на 20% от номинального значения. Например, если установлена частота 10 FPS, система должна обрабатывать в среднем не менее 8 кадров в секунду на протяжении длительного сеанса работы. Нестабильность частоты кадров, выражающаяся в резких спадах или нерегулярных интервалах между обработкой, может негативно сказаться на восприятии информации пользователем и снизить эффективность анализа динамических сцен. Это требование касается не только пиковой производительности, но и устойчивости системы под нагрузкой, при работе других приложений.

2. Надежность:

- система должна демонстрировать устойчивую работу без критических сбоев, приводящих к аварийному завершению процесса, зависанию или необходимости принудительной перезагрузки, в течение не менее 4 часов непрерывной работы. Данный временной интервал выбран как репрезентативный для типичного сеанса интенсивного использования. Надежность обеспечивается за счет качественного кода, обработки исключительных ситуаций на всех уровнях, стабильности используемых внешних библиотек и API, а также тщательного тестирования в условиях, приближенных к реальным.
- система должна корректно и предсказуемо обрабатывать внештатные ситуации, не инициированные пользователем напрямую. Ключевой сценарий — это ситуация, когда целевое окно для захвата закрывается, сворачивается, меняет свой размер или положение. В таких случаях система не должна аварийно завершаться, а либо автоматически приостанавливать захват с выводом информативного сообщения пользователю, либо, если настроен захват всего экрана, продолжать работу в штатном режиме. Поведение системы в подобных случаях должно быть четко определено и документировано.

3. Эффективность использования ресурсов:

- средняя загрузка центрального процессора системой в активном режиме (при запущенном процессе захвата и анализа) не должна превышать 15% на эталонном оборудовании, например, на процессоре среднего класса последних поколений. Это требование критично для фоновых и вспомогательных приложений, так как они не должны существенно мешать работе основного программного обеспечения, ради которого, собственно, и используется система анализа. Высокая нагрузка на CPU может вызывать замедление работы других программ, повышение

температуры компонентов и общее снижение комфорта пользователя.

- использование оперативной памяти системой должно быть стабильным и предсказуемым, не превышая установленного лимита в 500 МБ. Объем потребляемой памяти не должен иметь тенденции к неконтролируемому росту (утечке памяти) при длительной работе. Стабильное потребление ОЗУ важно для работы на компьютерах с ограниченными ресурсами и для поддержания общей отзывчивости операционной системы. Память должна эффективно использоваться для буферов кадров, данных модели ИИ и графических ресурсов оверлея, с своевременным освобождением неиспользуемых объектов.

4. Удобство использования:

- пользовательский интерфейс для настройки системы должен быть интуитивно понятным, логически организованным и не требовать от пользователя чтения объемной сложной документации для выполнения базовых операций, таких как выбор источника захвата, запуск анализа или изменение цвета оверлея. Элементы управления должны иметь понятные названия, поля ввода — валидацию и подсказки, а наиболее частые действия должны быть доступны минимальным количеством кликов. Это требование направлено на снижение порога входа и экономию времени пользователя.
- процесс установки и первоначальной настройки системы должен быть максимально упрощен и занимать у среднестатистического пользователя не более 10 минут. Это включает в себя распаковку дистрибутива (или запуск установщика), принятие лицензионного соглашения, выбор директории установки, настройку базовых параметров (например, горячих клавиш) и первичный запуск. Отсутствие необходимости в ручном редактировании

конфигурационных файлов, регистрации сложных системных компонентов или поиске дополнительных зависимостей напрямую влияет на доступность продукта для широкой аудитории.

5. Совместимость:

- система должна корректно работать под управлением операционной системы Windows 10 (64-битная версия) и выше, включая Windows 11. Это подразумевает отсутствие конфликтов с системными библиотеками, корректную работу с обновленными API и поддержку современных функций безопасности операционной системы. Тестирование должно проводиться на различных сборках и редакциях указанных ОС.
- система должна корректно функционировать в стандартной пользовательской среде при наличии активных антивирусных программ и систем мониторинга. Это означает, что ее работа не должна блокироваться как потенциально опасная активность без веских причин, а взаимодействие с системой на низком уровне (захват экрана, внедрение оверлея) должно осуществляться методами, не вызывающими ложных срабатываний у большинства распространенных средств защиты. В идеальном случае дистрибутив системы должен быть подписан цифровым сертификатом.

ЗАКЛЮЧЕНИЕ

В рамках выполнения научно-исследовательской работы был проведен всесторонний и детальный анализ актуальной и сложной проблемной области, связанной с проектированием и созданием программных систем для автоматического анализа экранного контента в реальном времени с применением передовых технологий искусственного интеллекта и инновационных методов вывода результатов в виде интерактивного графического наложения. Данная проблематика находится на стыке нескольких интенсивно развивающихся дисциплин — системного программирования, компьютерного зрения, машинного обучения и компьютерной графики, что обуславливает ее комплексный характер и высокую практическую значимость. В результате проведенного глубокого исследования были тщательно систематизированы, классифицированы и критически оценены существующие на текущий момент методологические подходы и технологические решения, применяемые для решения данной проблемы, как в зарубежной, так и в отечественной научно-технической практике. Были рассмотрены и подвергнуты сравнительному анализу различные подходы к реализации захвата экрана, начиная от классических, но ограниченных в производительности методов через устаревший Graphics Device Interface (GDI), и заканчивая современными высокопроизводительными техниками, использующими низкоуровневые возможности DirectX Graphics Infrastructure (DXGI) и специализированного аппаратного ускорения, предоставляемого ведущими производителями графических процессоров. Параллельно с этим были детально изучены и оценены с точки зрения компромисса между скоростью и точностью современные архитектуры сверточных нейронных сетей для базовых задач компьютерного зрения, таких как классификация, детекция объектов и семантическая сегментация, которые являются оптимальными для работы в условиях жестких временных ограничений, характерных для систем реального

времени. Кроме того, был проведен углубленный анализ возможных альтернативных способов интеграции графического оверлея в отображаемую пользователем сцену, спектр которых простирается от относительно простых в реализации, но не всегда надежных прозрачных окон верхнего уровня до сложных, требующих глубоких специфических знаний, методов внедрения в графический конвейер рендеринга целевого приложения через перехват системных вызовов. На основе проведенного всестороннего анализа был разработан и формализован пошаговый алгоритм рационального выбора адекватных средств решения, который структурированно учитывает ключевые взаимосвязанные параметры конкретной задачи: целевую аппаратно-программную платформу развертывания, строгие требования к общей задержке конвейера обработки, необходимую глубину и детализацию семантического анализа изображения, а также допустимую нагрузку на вычислительные ресурсы системы.

Важнейшим и наиболее существенным итогом проделанной работы стало комплексное формирование полного и непротиворечивого набора функциональных и нефункциональных требований к перспективному программному решению. Разработанные функциональные требования охватывают и детально описывают полный замкнутый цикл работы будущей системы: от гибко настраиваемого пользователем захвата произвольной области экрана и корректной загрузки предобученной модели искусственного интеллекта из внешнего хранилища до сложной интерпретации численных результатов работы нейронной сети и их наглядной визуализации через многофункциональный и конфигурируемый графический оверлей. Сформулированные нефункциональные требования, в свою очередь, задают четкие количественные метрики и качественные рамки для последующей этапов разработки, фокусируясь на таких критически важных атрибутах качества, как высокая производительность и отзывчивость, бесперебойная надежность и отказоустойчивость, эффективное использование процессорных ресурсов и оперативной памяти, а также простота освоения и удобство для

конечного пользователя. Совокупность этих требований формирует детальное техническое задание и задает однозначные, проверяемые критерии для всей последующей проектной, конструкторской и разработческой деятельности, минимизируя риски недопонимания между этапами проектирования и реализации.

Таким образом, проведенное фундаментальное исследование создает прочный теоретический базис и комплексный методологический задел для плавного и обоснованного перехода к следующей, практической стадии работ — непосредственной разработке, реализации, интеграции и всестороннему тестированию работоспособного программного комплекса в рамках выпускной квалификационной работы. Дальнейшая деятельность будет логически сосредоточена на детальном проектировании и документировании модульной архитектуры системы на основе выбранного и обоснованного принципа разделения ответственности, окончательном выборе конкретных инструментальных средств, библиотек и фреймворков из рассмотренных классов, а также на последовательной, поэтапной реализации и отладке каждого из ключевых модулей — захвата, анализа и визуализации — с их последующей интеграцией в единый исполняемый комплекс. Завершающим этапом станет глубокая оптимизация всего конвейера обработки данных, направленная на гарантированное достижение заданных в требованиях целевых показателей производительности, стабильности и потребления ресурсов, что в конечном итоге позволит продемонстрировать жизнеспособность предложенного подхода в целом.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. You Only Look Once: Unified, Real-Time Object Detection // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). – 2016.
2. Wu, Y., Kirillov, A., Massa, F., Lo, W.-Y., & Girshick, R. Detectron2. – Facebook AI Research, 2019. – URL: <https://github.com/facebookresearch/detectron2>
3. Microsoft. DirectX Graphics Infrastructure (DXGI) documentation. [Электронный ресурс]. – Microsoft Docs. – URL: <https://docs.microsoft.com/en-us/windows/win32/direct3ddxgi/dx-graphics-dxgi>
4. OBS Project. Open Broadcaster Software. [Электронный ресурс]. – URL: <https://obsproject.com/>
5. NVIDIA. NVIDIA Frame Buffer Capture (NVFBC) and NVIDIA Image Capture (NVIFR) documentation. [Электронный ресурс]. – NVIDIA Developer. – URL: <https://developer.nvidia.com/nvidia-capture-sdk> (дата обращения: 13.10.2024).
6. Paszke, A., Gross, S., Massa, F., et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library // Advances in Neural Information Processing Systems 32 (NeurIPS 2019). – 2019.
7. Abadi, M., Barham, P., Chen, J., et al. TensorFlow: A System for Large-Scale Machine Learning // 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI '16). – 2016.
8. ONNX Runtime. [Электронный ресурс]. – URL: <https://onnxruntime.ai/> (дата обращения: 14.10.2024).
9. Bradski, G. The OpenCV Library // Dr. Dobb's Journal of Software Tools. – 2000.

- 10.Осипов, А.В., Кремлев, А.С. Методы обработки видеопотоков в реальном времени для задач технического зрения // Информационные технологии и вычислительные системы. – 2018. – № 3. – С. 45-58.
- 11.Васин, С.А., Игнатов, Д.И. Сравнительный анализ современных архитектур сверточных нейронных сетей для детектирования объектов // Труды института системного программирования РАН. – 2020. – Т. 32, № 4. – С. 123-140.
- 12.Bochkovskiy, A., Wang, C.-Y., Liao, H.-Y.M. YOLOv4: Optimal Speed and Accuracy of Object Detection // arXiv preprint arXiv:2004.10934. – 2020.
- 13.Tan, M., Pang, R., Le, Q.V. EfficientDet: Scalable and Efficient Object Detection // Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). – 2020.
- 14.Microsoft. Desktop Duplication API. [Электронный ресурс]. – Microsoft Docs.–URL: <https://docs.microsoft.com/en-us/windows/win32/direct3ddxgi/desktop-dup-api> (дата обращения: 2020.09.15).
- 15.Кротов, Д.С. Принципы построения систем наложения графической информации на видеопоток в реальном времени // Программная инженерия. – 2019. – № 10(8). – С. 367-375.